PR Review

Do / Say / See

Local VS Code review, then GitHub PR review, chat, an AI fix and documentation.

4 pillars | 17 core pages | limit 20

The local review uses demo-site-pr-examples / codex/bulk-checkout-flagship. The browser chapters use demo-site-pr-recordings PR #8; the documentation result is PR #9. Both repositories are intentional.

Read each grouped step top to bottom. Do contains the clicks and scrolls; read only Say aloud. See is a frame from the delivered video. Times refer to that video. The narration is preserved in full, including the recorded opening and closing.

Creation, Send and approval actions describe the recorded flow. Rehearse with the linked existing results; use a separate training run when repeating those actions. Live pages may have changed. Resource links require the corresponding account access.

OPEN THE DEMO

Example repository

Recording repository

Recording PR #8

Acceptance criteria: issue #7

Docstrings PR #9

PILLARS AND PAGE LIMITS

Introduction - 1 page

Code Exists. Review Now. - 4 steps / pages

PR Created. Walk the Summary & Findings - 4 steps / pages

Chat & Teach - 4 steps / pages

One-Click Fix - 4 steps / pages

The core count includes the opening page when present. This cover, operator prompts and resource directory are outside the core. Every pillar uses at most four grouped steps.

INTRODUCTION / OPENING / 00:00-00:23

Introduce the flow

DO

Show the section’s flow slide. Keep the full slide visible and highlight each pillar as it comes up in the introduction.

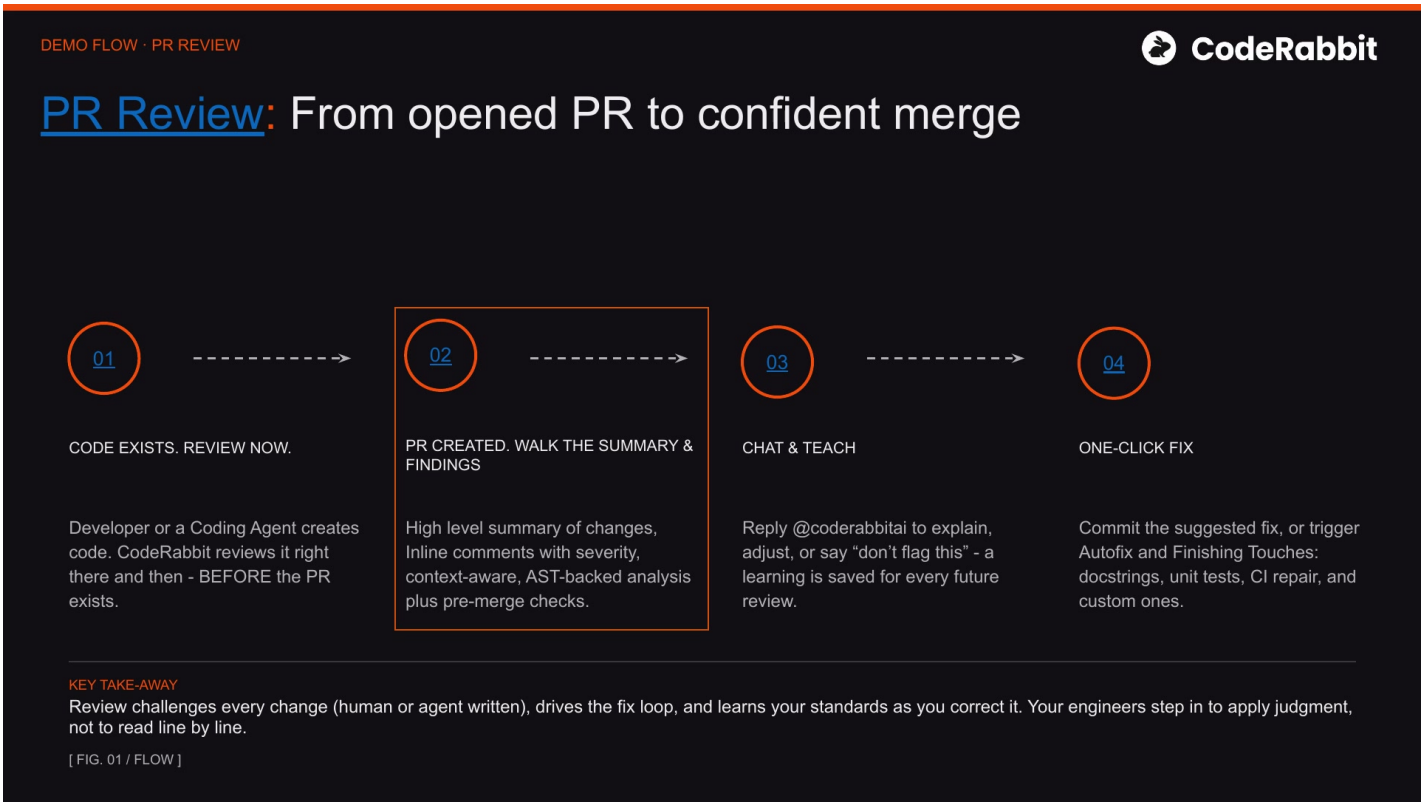
At the end of the introduction, switch to the first demo screen.

Slide deck

SAY

We'll start with code that a coding agent has already written. I'll review it locally in VS Code. Then open a pull request and walk through the summary and findings. We'll investigate a security issue and a discount bug, ask questions in the review, and teach CodeRabbit a business rule. Then we'll follow an AI generated fix through the code and tests, and look at the separate documentation change.

SEE / VIDEO FRAME AT 00:09



CODE EXISTS. REVIEW NOW. / STEP 1 OF 4 / 00:25-00:38

Clone and open the feature branch

DO

In Terminal, run cd ~/projects, then git clone https://github.com/coderabbit-demo/demo-site-pr-examples.git.

Run cd demo-site-pr-examples, then git checkout codex/bulk-checkout-flagship.

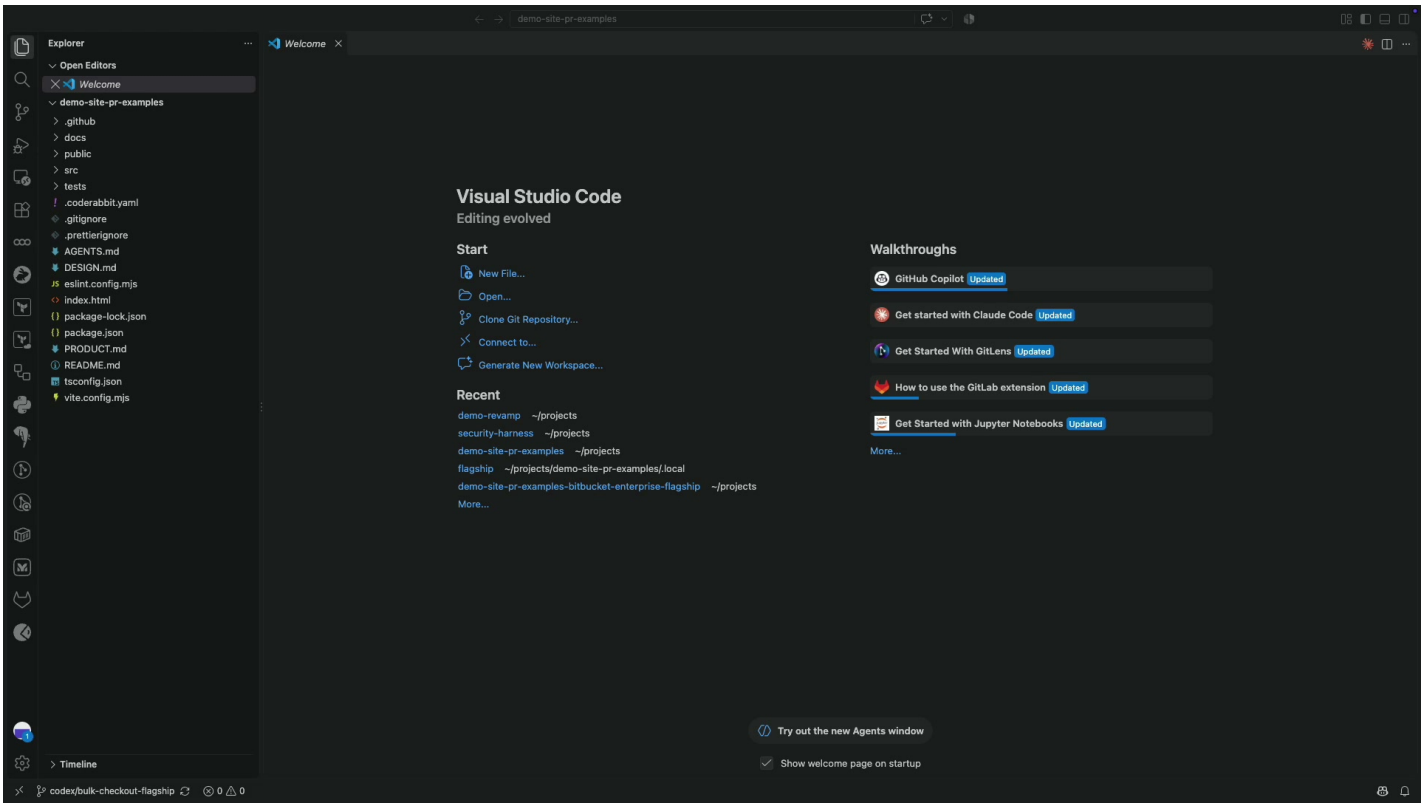
Run code . and switch to the VS Code window.

Example repository

SAY

In my Mac terminal, I'm in the projects folder. I clone the repository from GitHub and select the branch containing the implementation. Then I launch VS Code from this folder. This is my first look at the code the agent generated.

SEE / VIDEO FRAME AT 00:37



CODE EXISTS. REVIEW NOW. / STEP 2 OF 4 / 00:38-00:57

Set the scope and start the review

DO

Click the CodeRabbit icon in VS Code's activity bar. If needed, open Extensions, search CodeRabbit and open the installed extension first.

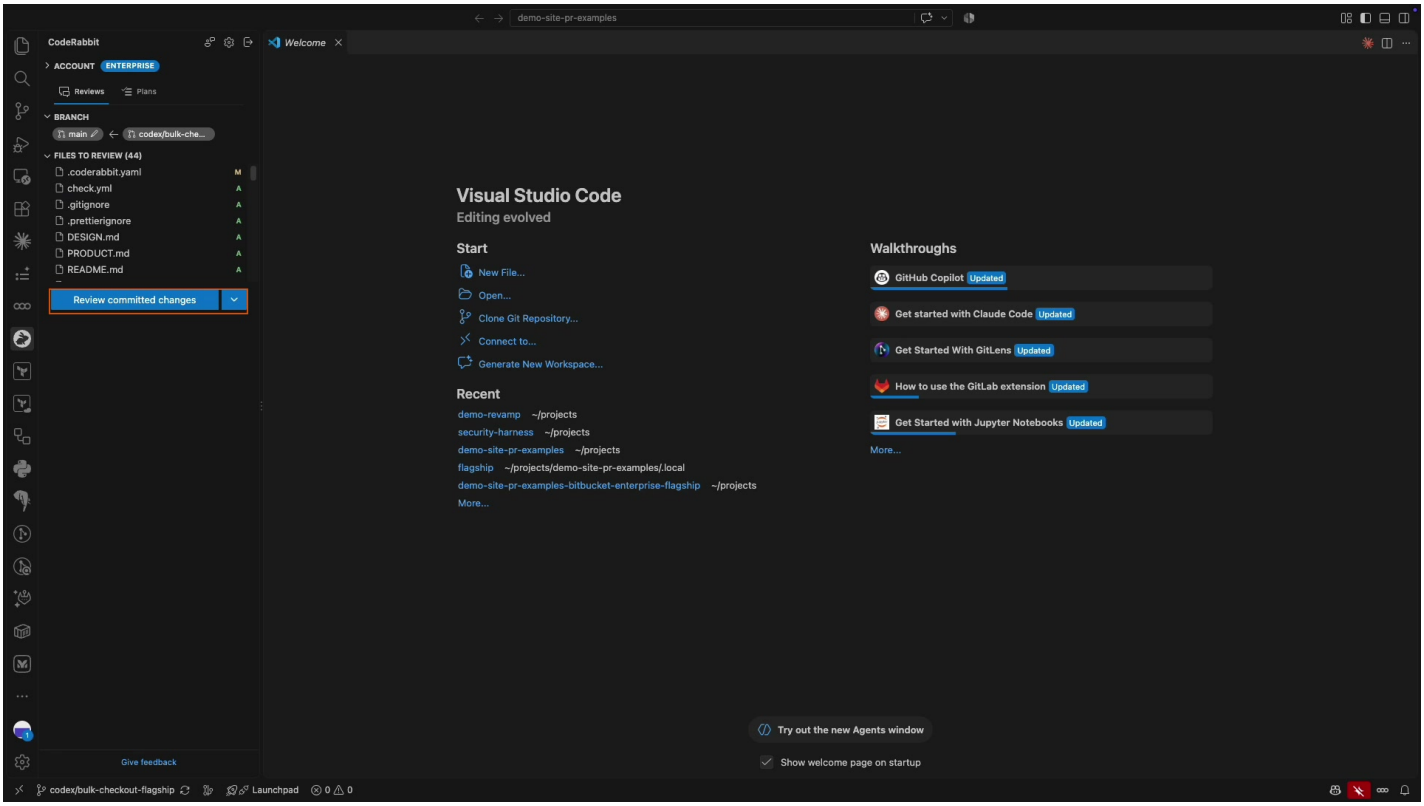
Click the base selector, type main and select main. Open the scope selector and choose Review committed changes.

Confirm the committed files are listed, then click Review committed changes. Wait for the findings.

SAY

I open the CodeRabbit extension so I can review the changes locally. The agent's work is already committed to this branch. I choose main as the base and select review committed changes. That compares this branch with main. Even with no pending edits, there are committed changes to review. I start the review.

SEE / VIDEO FRAME AT 00:52



CODE EXISTS. REVIEW NOW. / STEP 3 OF 4 / 00:57-01:26

Inspect the SQL finding

DO

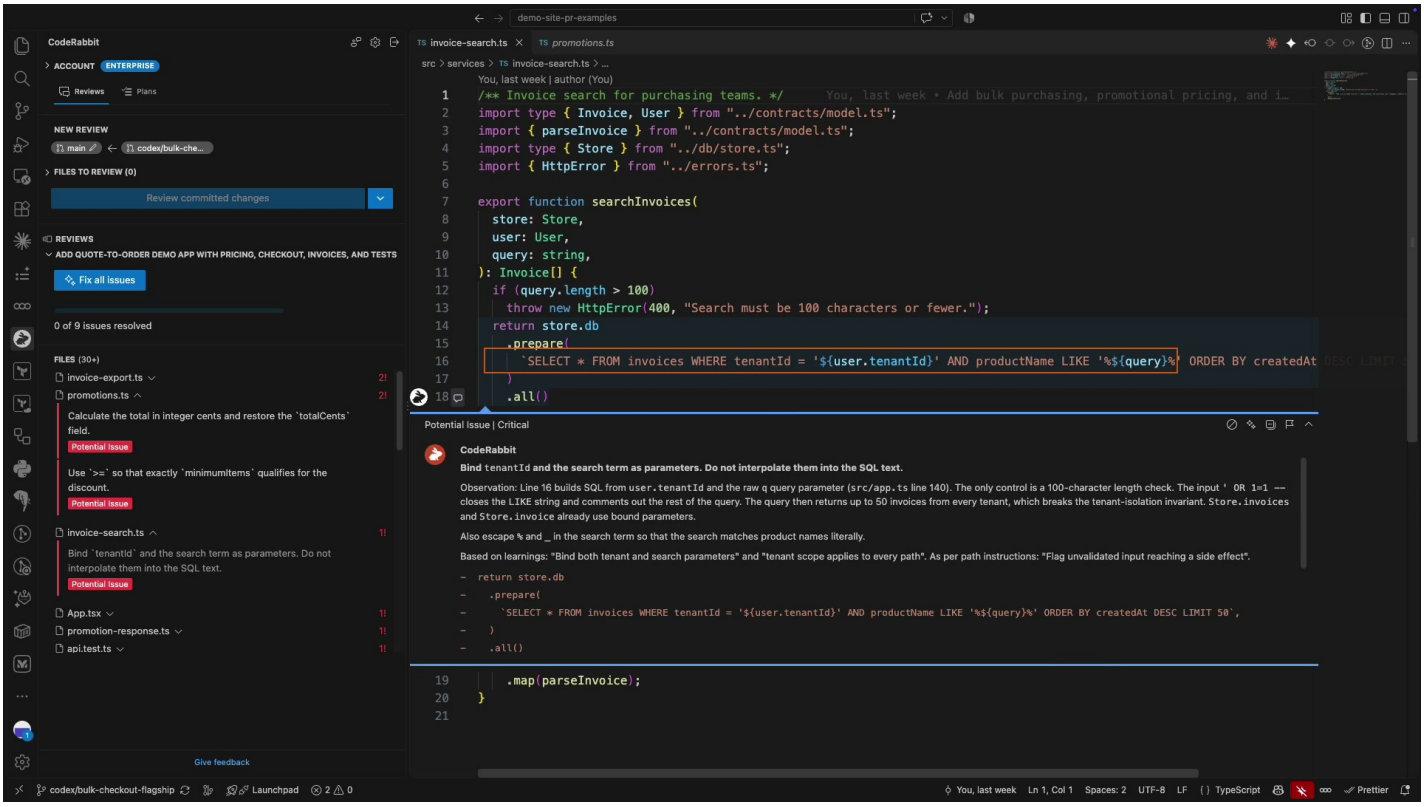
Click the invoice-search SQL injection finding in the CodeRabbit results.

Keep src/services/invoice-search.ts open. Scroll the inline explanation to the interpolated query, tenant boundary and parameter-binding recommendation.

SAY

When the results are ready, I open the security finding. CodeRabbit has identified a SQL injection issue in invoice search. Opening the finding takes me to the affected code and the explanation. Search text is being inserted directly into the database query which could let someone change the query and retrieve another customer's invoice. The agent generated tests and those tests passed. This review still found a security problem before I opened the pull request.

SEE / VIDEO FRAME AT 01:17



CODE EXISTS. REVIEW NOW. / STEP 4 OF 4 / 01:27-02:17

Inspect the discount and fix choices

DO

Click the promotions boundary finding. Scroll to Proposed fix to show > and >=.

Show Apply suggestion and Fix with AI, then move back to the review panel to show Fix all.

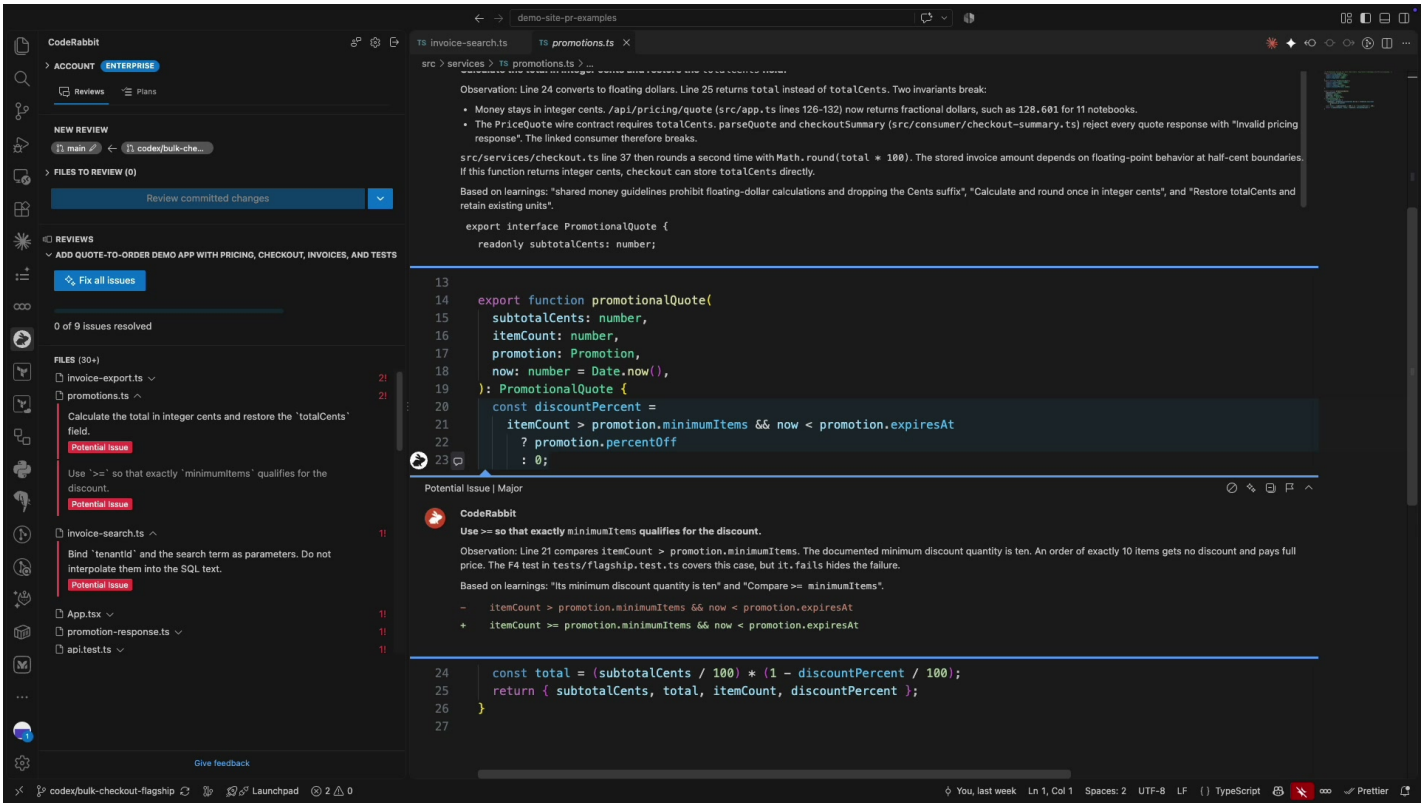
Leave the local changes unapplied and switch to the browser for the PR review.

Recording repository

SAY

Next I open the discount finding. The discount should apply at 10 items but the code checks for more than 10. An order of exactly 10 items pays full price. The suggested fix changes greater than to greater than or equal. That includes the minimum quantity as the business rule requires. The findings also give me ways to respond. I can apply a suggested fix for a simple change or use fix with AI to pass the finding and its context to my coding agent. Fix All lets me hand over the review findings together. After a fix, I would inspect the diff, run the relevant tests and review the updated code. I can do that work here on my branch before asking the team to review it. For this walkthrough, I'll carry the example into a pull request so we can see the team's review experience.

SEE / VIDEO FRAME AT 02:13



PR CREATED. WALK THE SUMMARY & FINDINGS / STEP 1 OF 4 / 02:18-02:55

Open the PR and read its summary

DO

For the existing run, open Recording PR #8. The recorded creation used main as base and codex/bulk-checkout-flagship as compare, then Create pull request.

In Conversation, open the CodeRabbit summary and expand Walkthrough. Scroll to Changes and Pre-merge checks.

Expand the checks and show Security Gate and the linked-issue assessment.

Recording PR #8

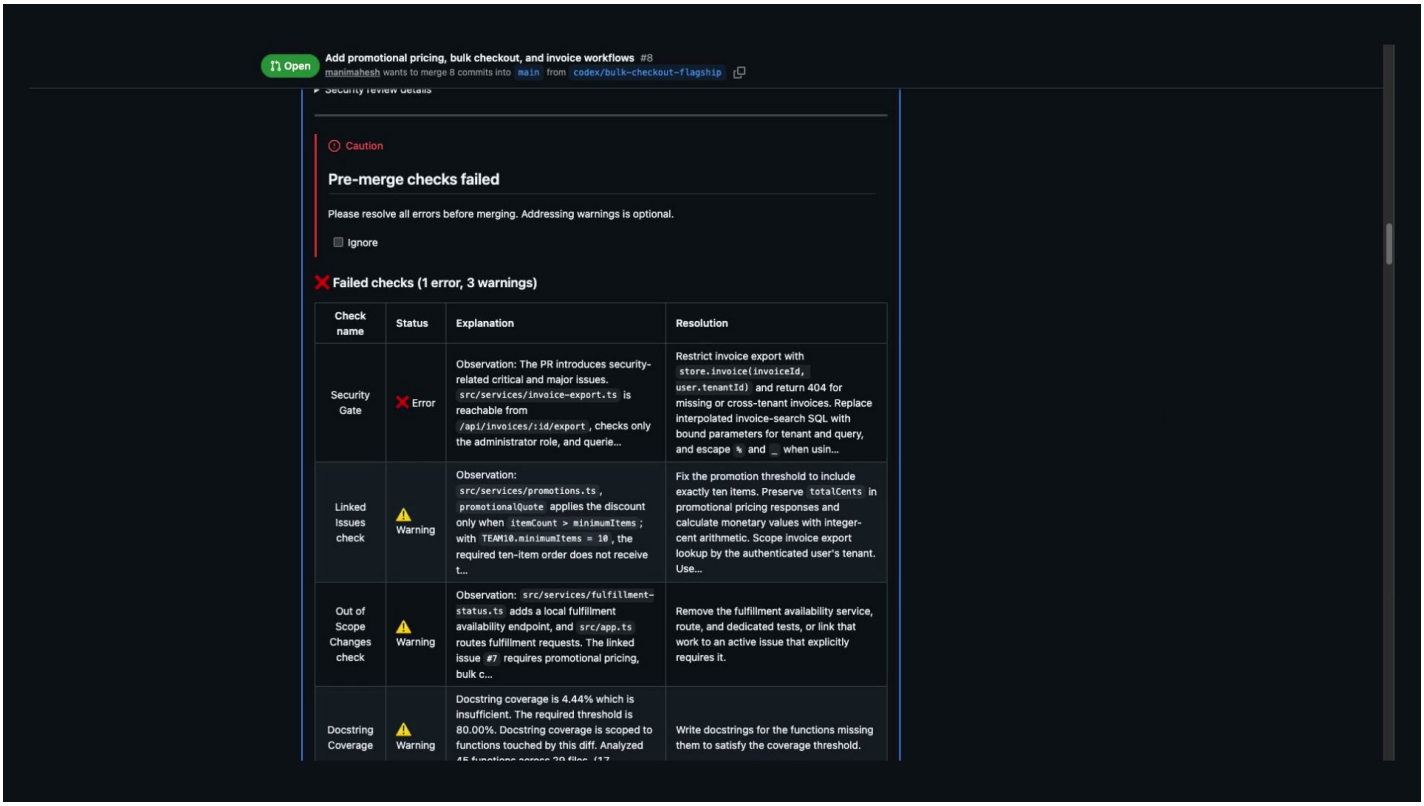
Acceptance criteria: issue #7

PR #8: CodeRabbit summary

SAY

I am now on GitHub opening a pull request against main. The description links the requirements for promotional pricing, checkout and invoice search and export. Once the pull request is created, CodeRabbit starts its review. I open the walkthrough first. It summarizes the implementation and the areas it touches so I can understand the scope before reading individual findings. Next, I scroll to the pre -merged checks. Security gate has failed because this review found major and critical security issues. The linked issues check also flags acceptance criteria the implementation has missed, including the discount threshold.

SEE / VIDEO FRAME AT 02:49



PR CREATED. WALK THE SUMMARY & FINDINGS / STEP 2 OF 4 / 02:56-03:20

Read the security evidence

DO

Open the SQL comment link. Expand Show outdated or the collapsed thread if necessary.

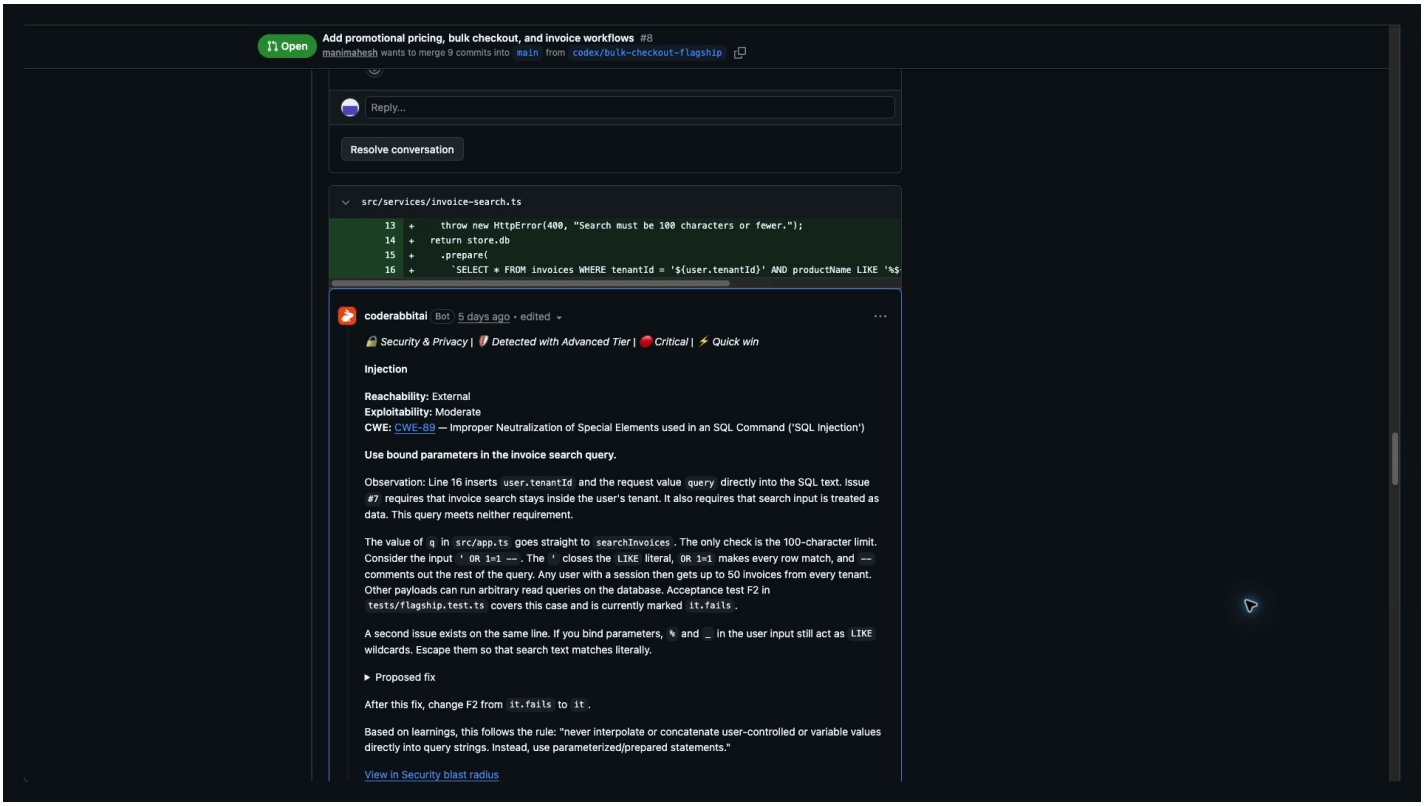
Scroll through the query and explanation. Expand Proposed fix and show tenant and search values supplied separately.

PR #8: SQL finding

SAY

I follow the security finding to the invoice search query. It's the same problem we saw locally. Search text goes straight into the SQL command. The finding identifies the affected line and explains how crafted input could expose invoices outside the customer's account. The proposed correction uses parameters so the database receives the search text as a value separate from the command.

SEE / VIDEO FRAME AT 03:02



PR CREATED. WALK THE SUMMARY & FINDINGS / STEP 3 OF 4 / 03:21-03:31

Check the exact-minimum discount

DO

Open the discount finding. Expand the thread and Proposed fix.

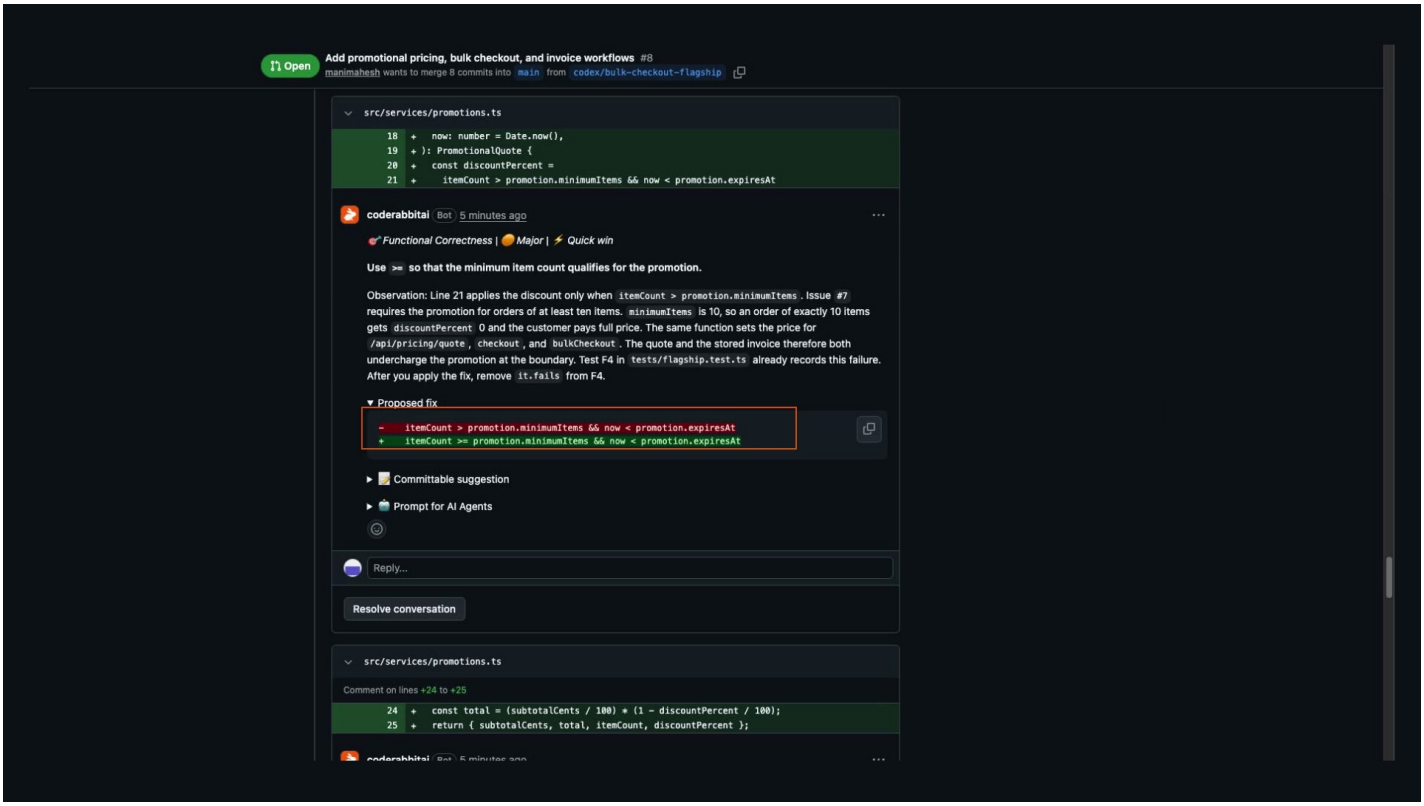
Keep the strict > condition and proposed >= correction visible together.

PR #8: discount finding

SAY

The discount finding is here too. Exactly 10 items should qualify and the current comparison excludes them. I can follow the explanation directly to the condition that needs to change.

SEE / VIDEO FRAME AT 03:29



PR CREATED. WALK THE SUMMARY & FINDINGS / STEP 4 OF 4 / 03:32-04:04

Check the consumer and review configuration

DO

Open the linked-consumer finding. Scroll to the total/totalCents mismatch and expand Sources or Linked repositories.

Read the src/client.ts and src/checkout.ts references. Then return to the recorded review and scroll to its review configuration/details.

Show Request Changes enabled. Read this alongside the repository's merge requirements.

PR #8: linked-consumer finding

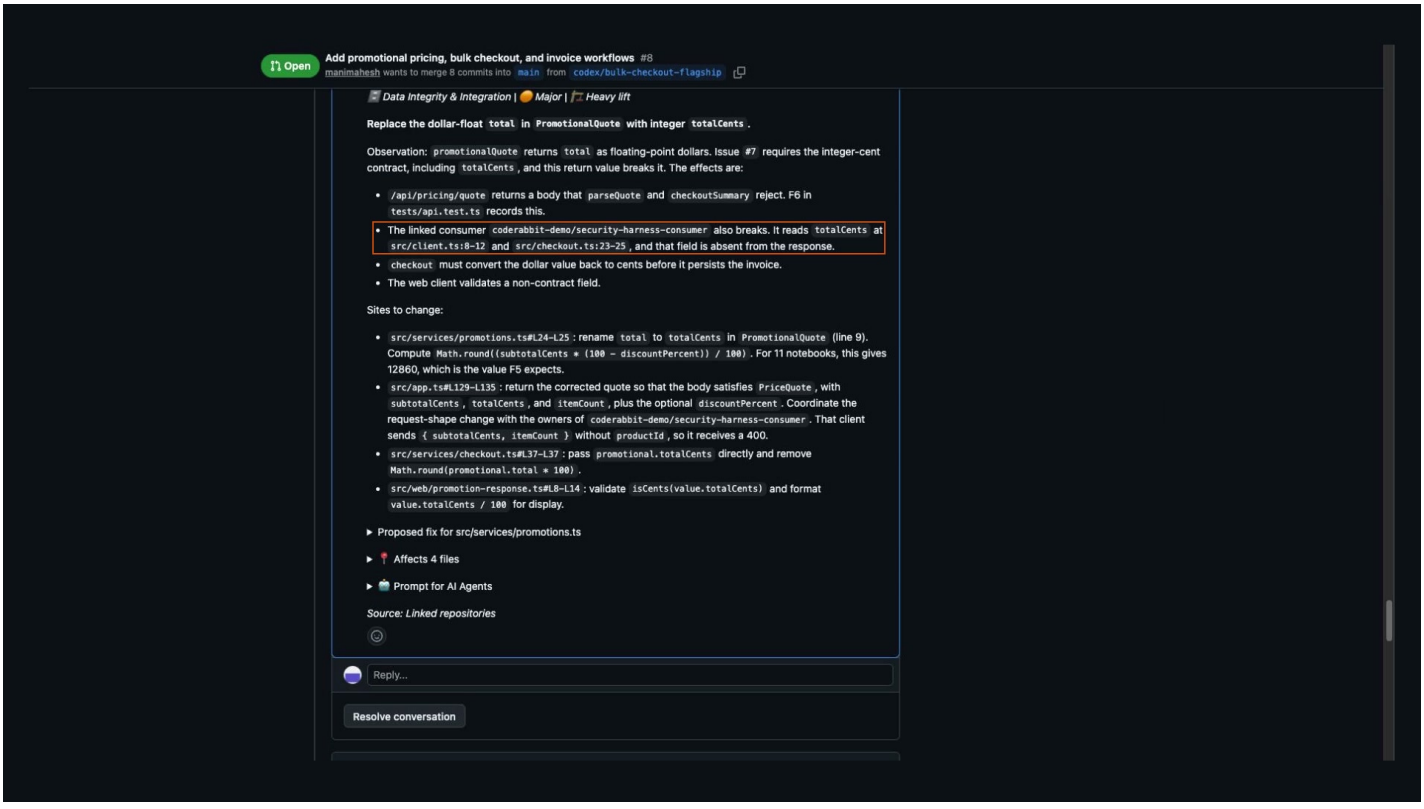
Linked consumer repository

PR #8: recorded review

SAY

There's also a compatibility issue across repositories. This change returns a field called total while a linked consumer expects totalCents. CodeRabbit points to the consumer code that depends on that contract. That gives me a concrete reason to investigate even if this application's own screen appears to work. Finally, I check the review configuration. Request changes is enabled. Whether those requests prevent a merge depends on the repository's branch rules, which we need to check alongside the review results.

SEE / VIDEO FRAME AT 03:40



CHAT & TEACH / STEP 1 OF 4 / 04:06-04:23

Add the paid-item rule

DO

Open the discount thread and click Reply.

Mention @coderabbitai. Explain that exactly ten paid items qualify and free promotional items do not count; ask it to remember the rule without changing code. Click Comment for a training run.

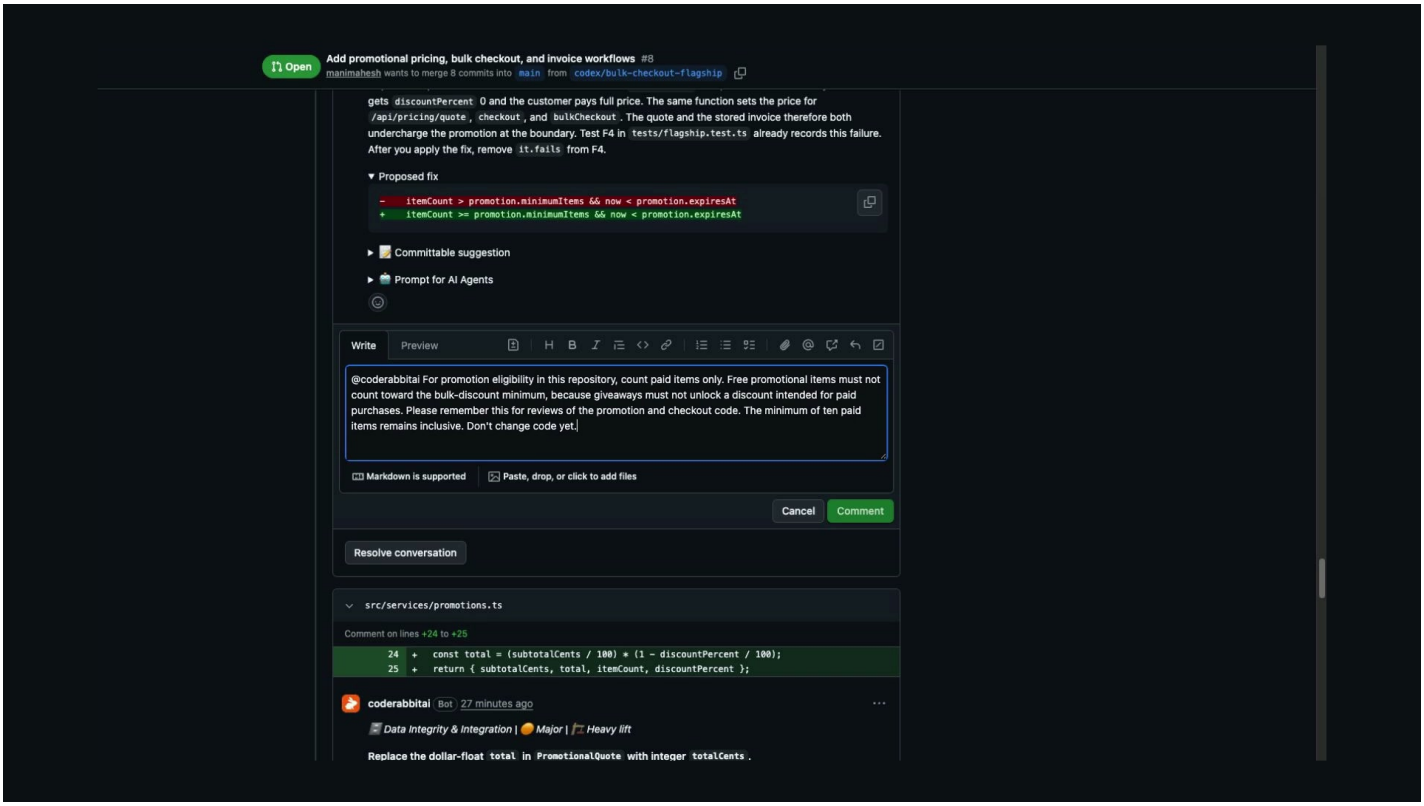
For rehearsal, open the recorded clarification and follow the existing response.

Discount clarification

SAY

I open the discount findings conversation to add a business rule. Exactly 10 paid items should qualify for the discount. Free promotional items should not count toward that minimum. I explain the distinction and ask CodeRabbit to remember it. At this point, I'm asking you to understand the rule before changing the code.

SEE / VIDEO FRAME AT 04:22



CHAT & TEACH / STEP 2 OF 4 / 04:24-04:42

Inspect the reply and learning

DO

Scroll to the reply confirming the inclusive paid-item minimum.

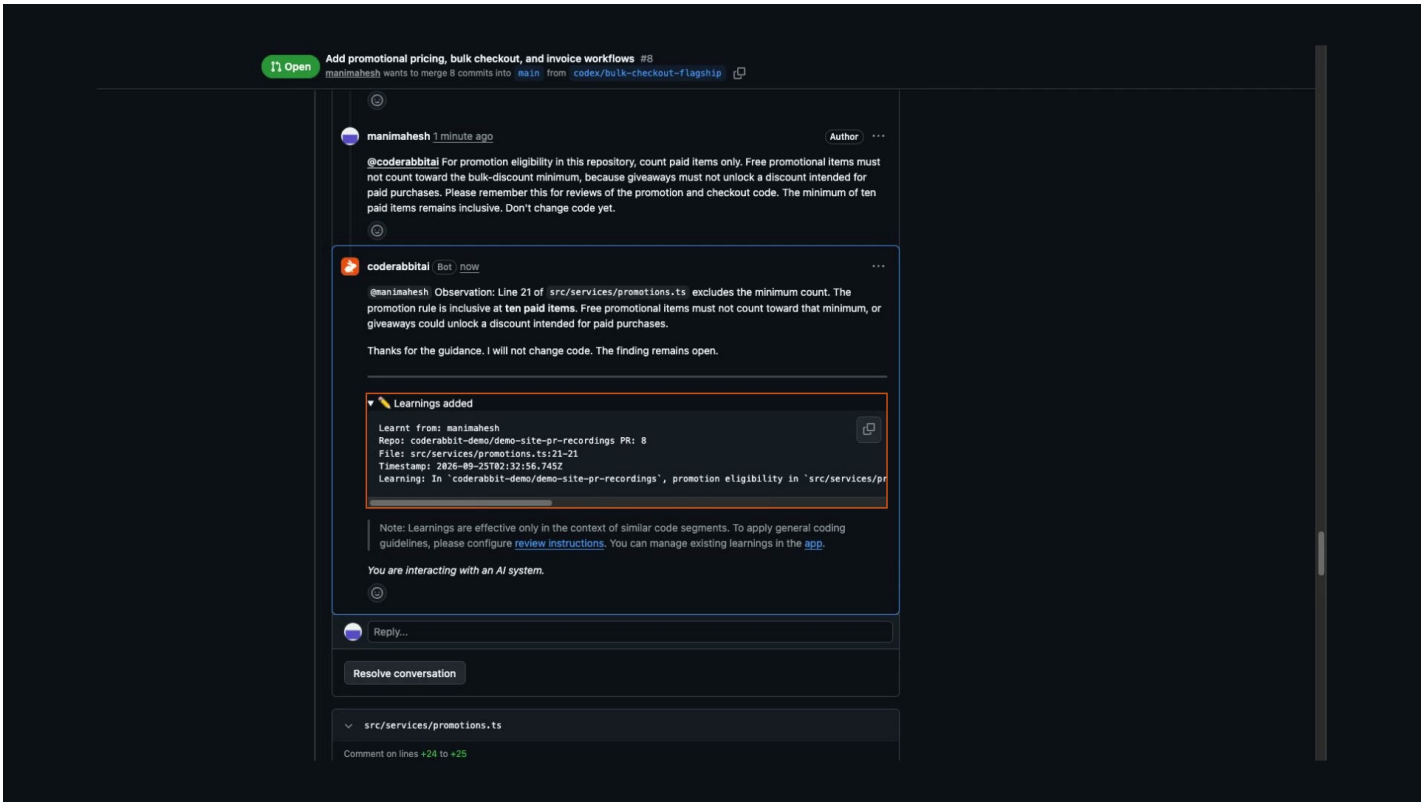
Expand Learnings added and read the saved rule and scope. Keep the finding open.

Discount response and learning

SAY

The reply confirms that eligibility depends on paid items and that exactly 10 paid items qualify. It also confirms that free promotional items do not count toward the minimum. The rule is captured as a learning. That gives later eligible reviews additional context about how this team expects the feature to behave.

SEE / VIDEO FRAME AT 04:41



CHAT & TEACH / STEP 3 OF 4 / 04:43-04:57

Ask for a security explanation

DO

Open the SQL thread and click Reply.

Enter: @coderabbitai, explain this security issue in simple terms and also quote examples where this was exploited in the real world.

Click Comment for a training run and wait. For rehearsal, open the saved question and response.

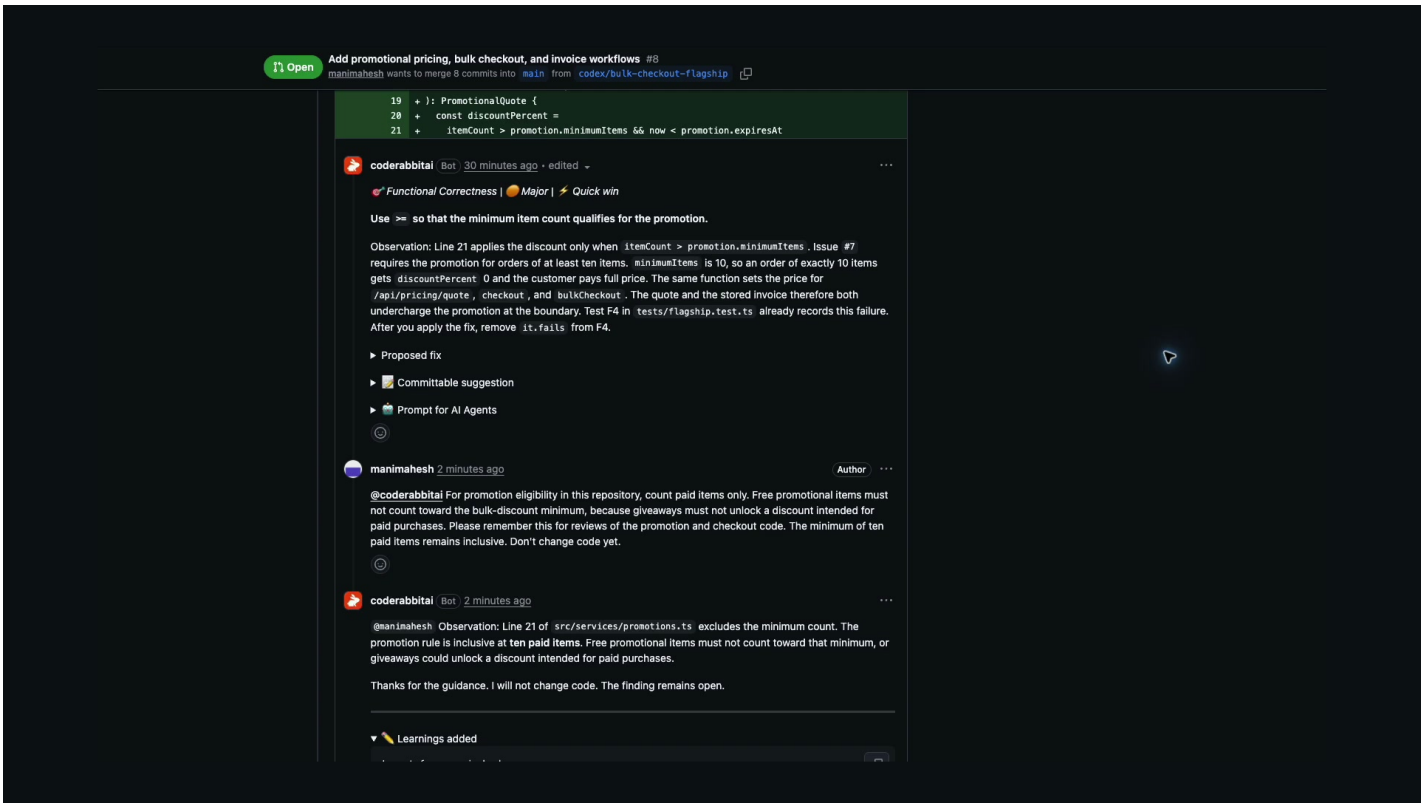
Security question

Security explanation and examples

SAY

Now I return to the SQL injection finding. I ask CodeRabbit to explain the problem in plain language and give examples of this type of vulnerability. The answer uses an ordinary search to explain the risk. A word like laptop should be treated as text.

SEE / VIDEO FRAME AT 04:56



CHAT & TEACH / STEP 4 OF 4 / 04:58-05:28

Read the explanation and examples

DO

Scroll the security response to crafted input and bound parameters.

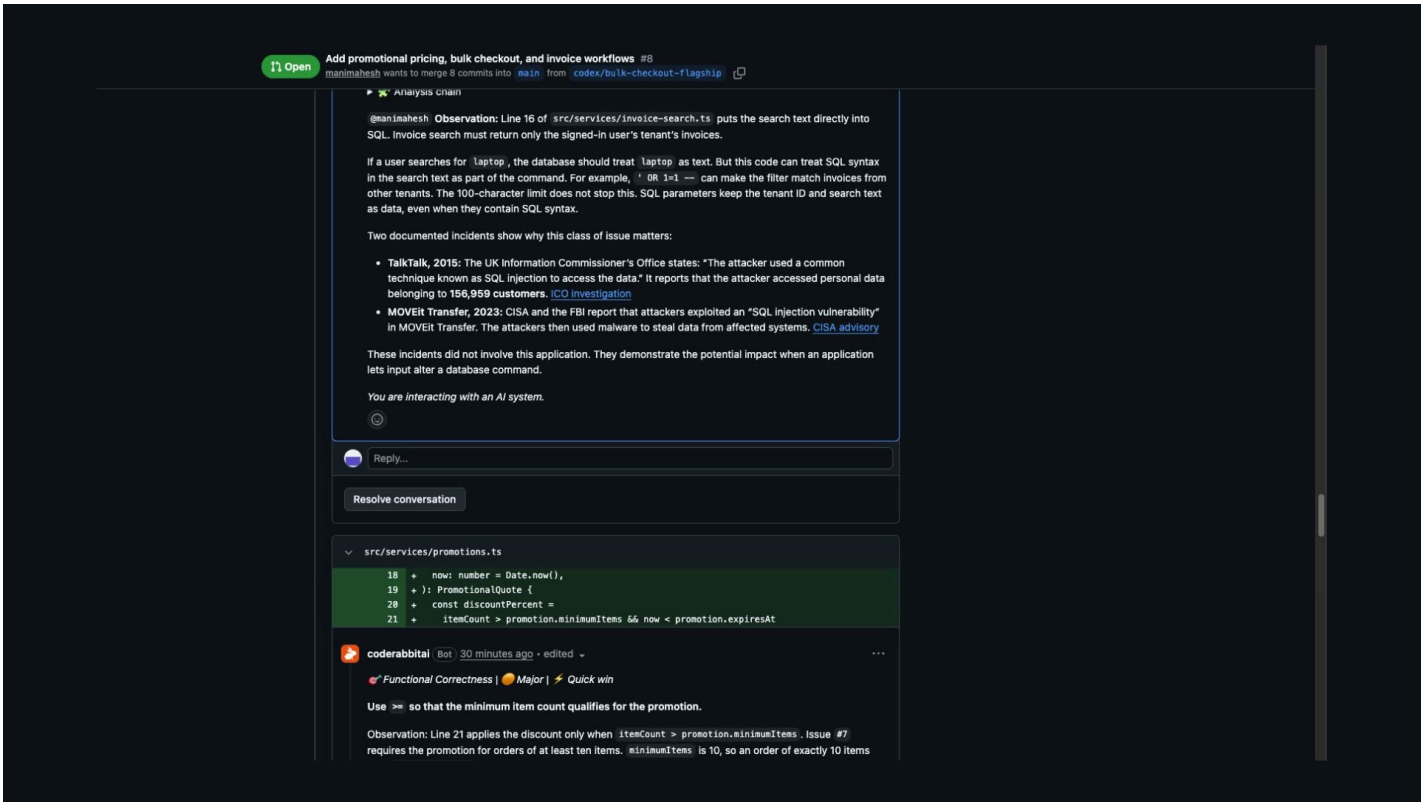
Continue to the TalkTalk and MOVEit examples and their citations. Keep the reply in the original finding's thread for the closing explanation.

Security explanation and examples

SAY

In this query, specially crafted text can change the database command and expose another customer's invoices. It also explains why a character limit does not fix the problem and why parameters keep search values separate from the command. The reply includes examples of the same vulnerability class in other systems. I can ask a follow up here with the relevant code and the original finding still in view. That helps me understand what the fix needs to protect before I request a change.

SEE / VIDEO FRAME AT 05:26



ONE-CLICK FIX / STEP 1 OF 4 / 05:29-05:38

Request the focused fix

DO

Return to the discount finding and click Reply.

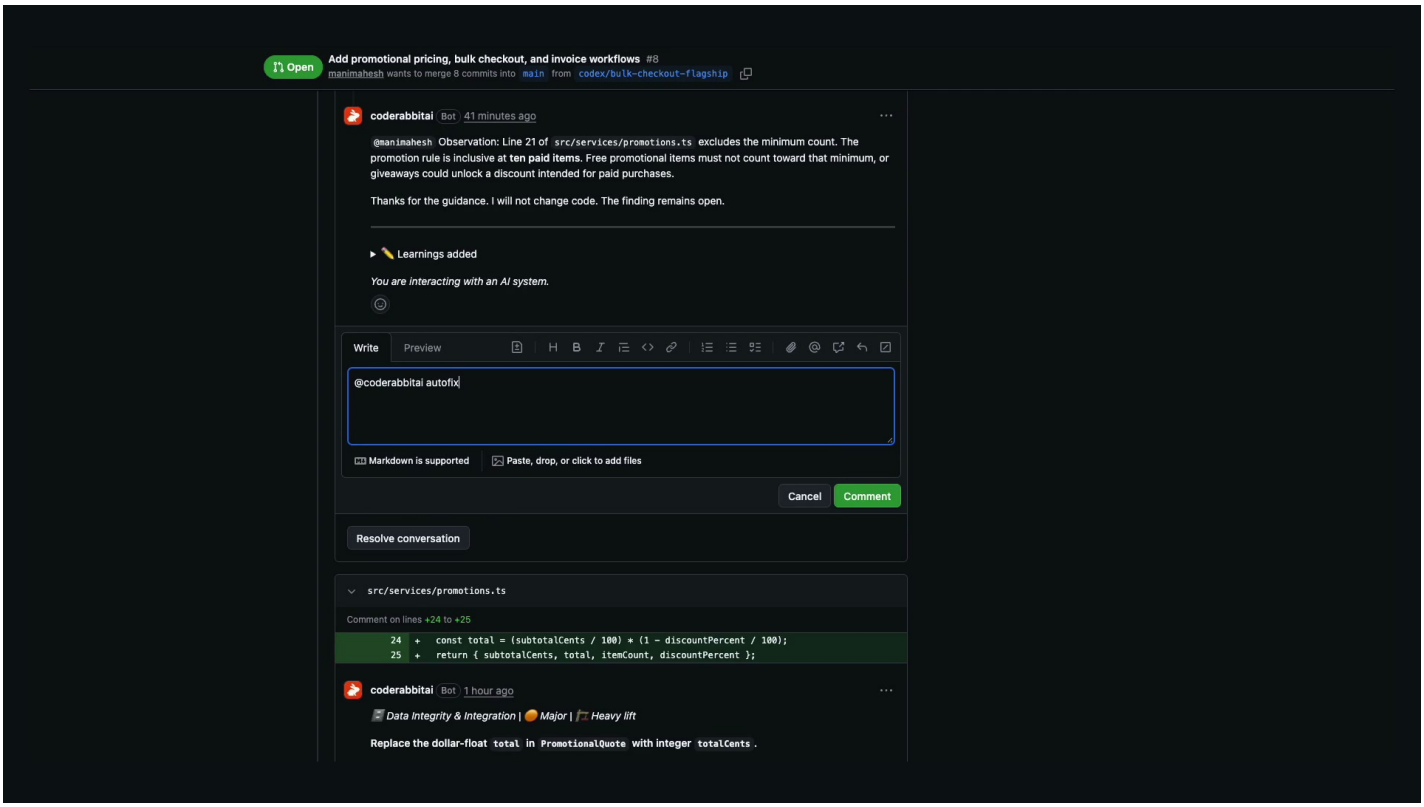
Type @coderabbitai autofix and click Comment for a training run. For rehearsal, open the recorded request and wait-state response.

Recorded autofix request

SAY

I return to the discount finding and ask AI to fix it from the review thread. The request includes the affected code and the explanation of the boundary problem.

SEE / VIDEO FRAME AT 05:36



ONE-CLICK FIX / STEP 2 OF 4 / 05:38-06:11

Inspect the generated change and tests

DO

Open the generated commit. Scroll to src/services/promotions.ts and tests/flagship.test.ts.

Show > becoming >= and it.fails becoming a normal test. Check below/at/above-minimum and expiration coverage.

The recorded operator validated the generated commit locally and applied that same commit to PR #8. Keep the patch visible while describing that result.

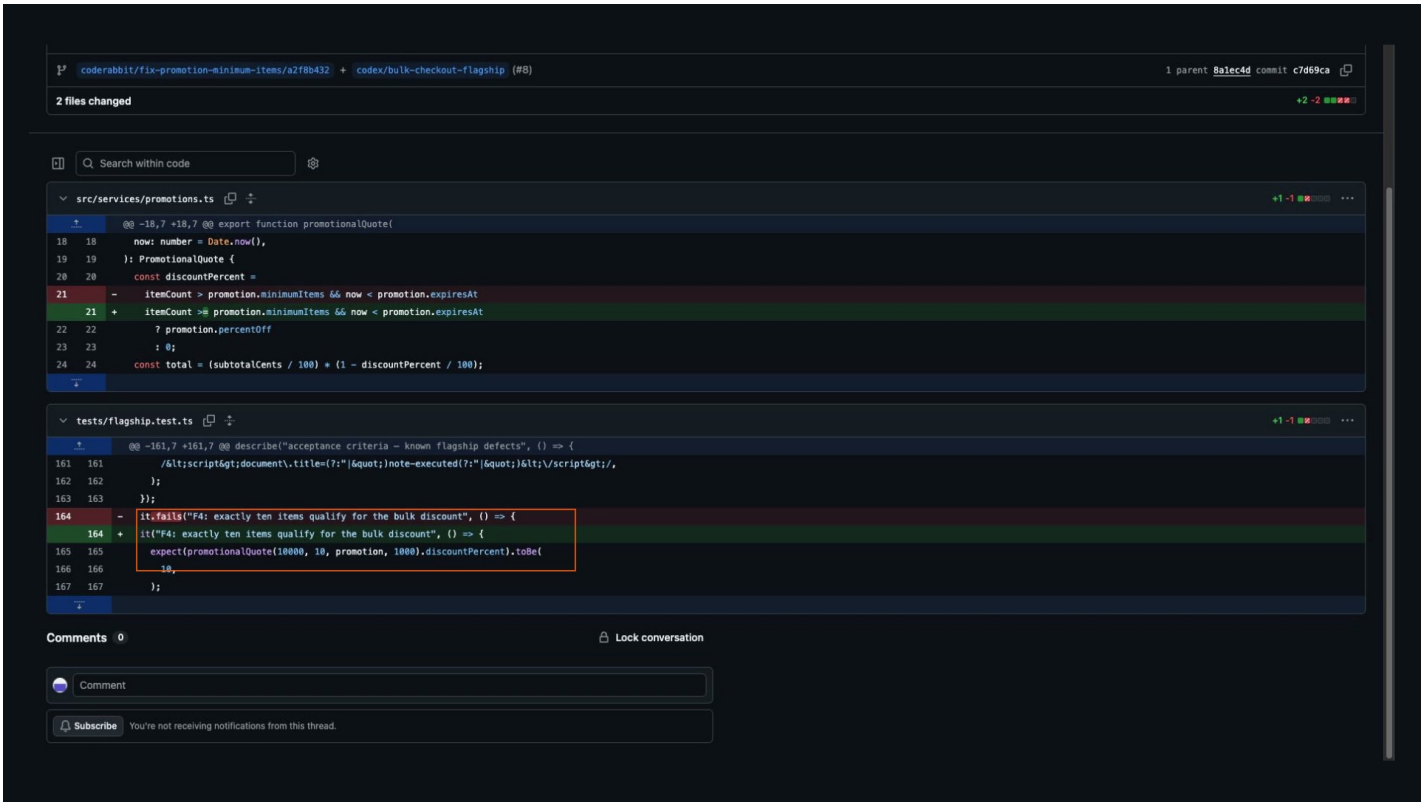
Discount-fix commit

Recording PR #8

SAY

When the generated change is ready, I open the diff. Greater than becomes greater than or equal, so an order of exactly 10 items now meets the minimum. The regression test for that boundary becomes a normal passing assertion. The focus checks cover quantities below, at and above the minimum. They also confirm that an expired promotion remains ineligible. After validating the generated commit, we apply that same commit to the pull request branch. This patch addresses the boundary bug. The other findings still need their own attention.

SEE / VIDEO FRAME AT 05:57



ONE-CLICK FIX / STEP 3 OF 4 / 06:12-06:26

Request and open the documentation PR

DO

Return to the CodeRabbit summary and expand Finishing Touches.

Select Generate Docstrings, then Create New PR for a training run; wait for completion.

For rehearsal, open Docstrings PR #9. Read the head/base branch line and description.

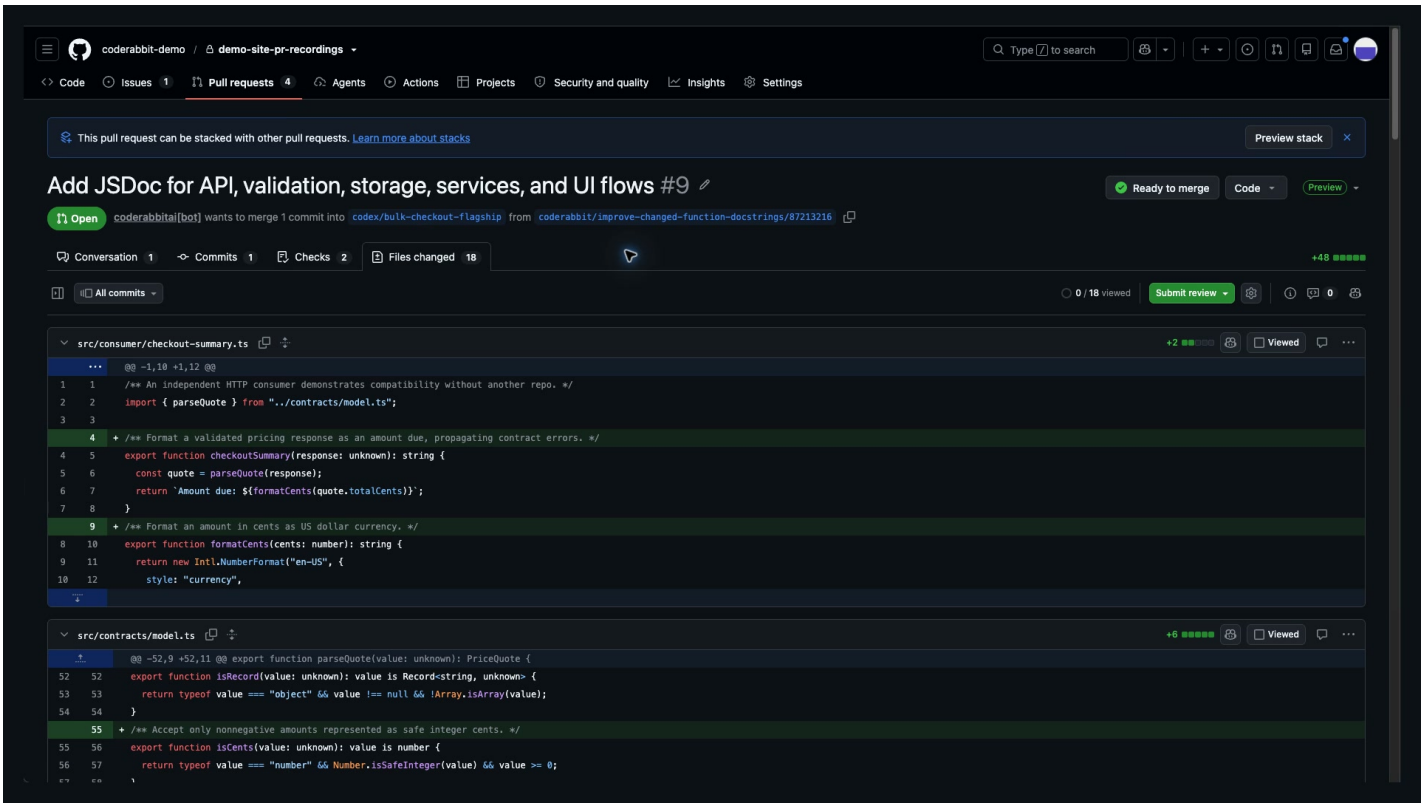
PR #8: CodeRabbit summary

Docstrings PR #9

SAY

There's another action I want to show. Under finishing touches, I select generate docstrings and choose create new PR. That prepares a separate documentation change for review. I open the resulting pull request and inspect the diff.

SEE / VIDEO FRAME AT 06:26



ONE-CLICK FIX / STEP 4 OF 4 / 06:26-07:03

Inspect documentation and finish

DO

Click Files changed on PR #9. Use the file list to open the quote service and request parser.

Scroll to the new quote and parseQuoteRequest comments and compare them with the functions.

Leave both PRs open. Return to the flow slide for the silent closing hold.

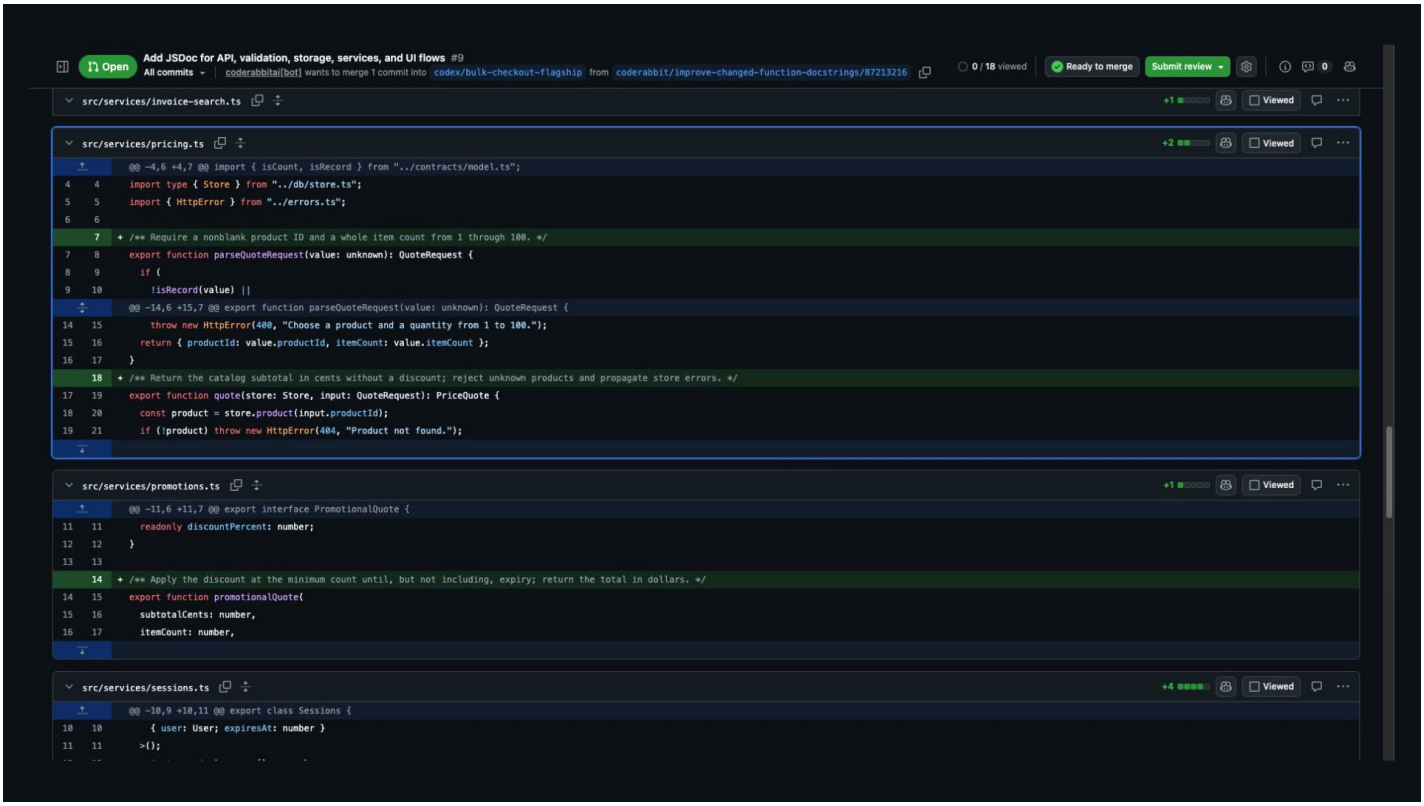
PR #9: files changed

Recording PR #8

SAY

The new comments explain what the functions expect, what they return, and how they handle errors. For this example, this docstring describes the catalog subtotal in cents and what happens when a product cannot be found. Beside it, the request parser documents the product ID and quantity requirements. We check that removing the comments leaves the code unchanged. I still read the descriptions against implementation before accepting them. We've reviewed the code locally, examined the pull request, clarified the findings, and inspected an AI generated fix. The resulting changes are ready for the team's review.

SEE / VIDEO FRAME AT 07:01



Resource directory

Links open the recorded examples. Historical comments and pinned source revisions preserve the relevant context when the live PR has moved on.

Example repository
https://github.com/coderabbit-demo/demo-site-pr-examples

Recording repository
https://github.com/coderabbit-demo/demo-site-pr-recordings

Recording PR #8
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8

Acceptance criteria: issue #7
https://github.com/coderabbit-demo/demo-site-pr-recordings/issues/7

Docstrings PR #9
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/9

Slide deck
https://docs.google.com/presentation/d/1zk6UD0ZoAxtGRqDTQL-MVulAWu5MH4nYmd-9xJBpU5Y/edit

PR #8: CodeRabbit summary
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8#issuecomment-5825402538

PR #8: SQL finding
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8#discussion_r4100325576

PR #8: discount finding
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8#discussion_r4100325582

PR #8: linked-consumer finding
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8#discussion_r4100325590

Linked consumer repository
https://github.com/coderabbit-demo/security-harness-consumer

PR #8: recorded review
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8#pullrequestreview-5312439610

Discount clarification
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8#discussion_r4100462792

Discount response and learning
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8#discussion_r4100465216

Security question
https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8#discussion_r4100474606

Security explanation and examples

https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8#discussion_r4100479733

Recorded autofix request

https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/8#discussion_r4100694595

Discount-fix commit

https://github.com/coderabbit-demo/demo-site-pr-recordings/commit/c7d69caa0ce797de95436f95c99176c983867dac

PR #9: files changed

https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/9/files